🏗️CSS Responsiveness

1. What is Responsiveness?

Responsive design means your website should look good and work properly on every screen size:

- Small mobile phones
- Tablets
- Laptops
- Large desktop monitors

Earlier, developers used to create separate mobile websites (`m.facebook.com` type).

Now modern websites use a single responsive layout that automatically adjusts according to screen size.

Core Parts of Responsive Design

Responsive design mainly depends on 3 things:

1. Fluid Layouts
2. Media Queries
3. Flexible Images & Media

---

2. CSS Units — The Foundation

Units are divided into two categories:

- Absolute Units
- Relative Units

2.1 Absolute Units

`px` (Pixel)

Fixed size unit.

```
.box {
  width:300px;
  font-size:16px;
}
```

The size stays the same on every screen.

When to Use `px`

Good for:

- Borders
- Small shadows
- Precise control

Avoid using `px` for major layouts.

2.2 Relative Units

These units change according to screen or parent size.

Percentage `%`

Calculated according to parent size.

```
.parent {
  width:800px;
}

.child {
```

```
  width:50%;
}
```

`50%` of `800px` = `400px`

`em`

Calculated according to the parent font-size.

```
.parent {
  font-size:20px;
}

.child {
  font-size:1.5em;
}
```

Result:

```
20px × 1.5 = 30px
```

Problem with `em`

Nested `em` values compound and become difficult to manage.

Better for:

- Padding
- Margin
- Gap

`rem` (Root em)

Calculated according to the root (`html`) font-size.

Default:

```
html {
  font-size:16px;
}
```

Example:

```
.heading {
  font-size:2rem;
}

.text {
  font-size:1rem;
}

.small {
  font-size:0.875rem;
}
```

Why `rem` is Best

- Easy scaling
- Better accessibility
- Entire website can scale from one root value

Pro Trick

```
html {
  font-size:62.5%;
}
```

Now:

```
1rem = 10px
```

Example:

```
.box {
  padding:2rem;
}
```

Result:

```
20px
```

`vw` and `vh`

Viewport based units.

```
1vw = 1% viewport width
1vh = 1% viewport height
```

Example:

```
.hero {
  width:100vw;
  height:100vh;
}
```

`vmin` and `vmax`

```
vmin = smaller of vw and vh
vmax = larger of vw and vh
```

Example:

```
.square {
  width:50vmin;
  height:50vmin;
}
```

`ch`

Based on character width.

```
.article {
  max-width:65ch;
}
```

Very useful for readable article widths.

`fr` (Grid Only)

Fraction unit in CSS Grid.

```
.grid {
  display:grid;
  grid-template-columns:1fr2fr1fr;
}
```

Unit Summary

Purpose	Best Unit
Font sizes	`rem`
Layout widths	`%`, `fr`, `vw`
Full screen sections	`vh`, `vw`
Borders	`px`
Readable text width	`ch`

3. Viewport Meta Tag

Must add this inside `<head>`:

```
<meta
name="viewport"
content="width=device-width, initial-scale=1.0"
/>
```

Why It is Important

Without this tag:

- Mobile browsers zoom out the website
- Media queries may fail
- Layout looks broken on phones

---

4. Media Queries

Media queries apply CSS based on screen conditions.

Basic Syntax

```
@media (max-width:768px) {
  .navbar {
    flex-direction:column;
  }

  .heading {
    font-size:1.5rem;
  }
}
```

`min-width` vs `max-width`

Desktop First (`max-width`)

```
.container {
  display:grid;
  grid-template-columns:1fr1fr1fr;
}

@media (max-width:768px) {
  .container {
    grid-template-columns:1fr1fr;
  }
}

@media (max-width:480px) {
  .container {
    grid-template-columns:1fr;
  }
}
```

Mobile First (`min-width`)

```
.container {
  display:grid;
  grid-template-columns:1fr;
}

@media (min-width:481px) {
  .container {
    grid-template-columns:1fr1fr;
  }
}

@media (min-width:769px) {
  .container {
    grid-template-columns:1fr1fr1fr;
  }
}
```

Why Mobile First is Better

- Better performance
- Cleaner CSS
- Easier scaling
- Mobile users are majority

Common Breakpoints

```
/* Mobile first */

@media (min-width:481px) {
/* Large phones */
}

@media (min-width:768px) {
/* Tablets */
}

@media (min-width:1024px) {
/* Small laptops */
}

@media (min-width:1280px) {
/* Desktops */
}

@media (min-width:1536px) {
/* Large desktops */
}
```

Media Query Operators

AND

```
@media (min-width:768px)and (max-width:1024px) {
```

```
}
```

OR

```
@media (max-width:480px), (min-width:1200px) {
}
```

Orientation

```
@media (orientation:landscape) {
}

@media (orientation:portrait) {
}
```

Hover Support

```
@media (hover:hover) {
  .button:hover {
    background:red;
  }
}
```

Dark Mode Detection

```
@media (prefers-color-scheme:dark) {
}
```

Reduced Motion

```
@media (prefers-reduced-motion:reduce) {
}
```

---

5. Flexbox for Responsive Layouts

Flexbox is mainly for 1D layouts.

`flex-wrap`

```
.cards {
  display:flex;
  flex-wrap:wrap;
  gap:1rem;
}

.card {
  flex:11300px;
}
```

Understanding `flex: 1 1 300px`

```
flex-grow   = 1
flex-shrink = 1
flex-basis  = 300px
```

Cards automatically wrap based on available space.

Responsive Navbar Example

```
.navbar {
  display:flex;
  flex-wrap:wrap;
  justify-content:space-between;
  align-items:center;
  padding:1rem;
}

.nav-links {
  display:flex;
  gap:1.5rem;
  flex-wrap:wrap;
}

@media (max-width:600px) {
  .nav-links {
    flex-direction:column;
    width:100%;
  }
}
```

6. CSS Grid — 2D Layouts

Grid works with rows and columns.

Famous Responsive Grid

```
.grid {
  display:grid;
  grid-template-columns:
repeat(auto-fit,minmax(250px,1fr));

  gap:1rem;
}
```

Understanding the Magic

```
auto-fit  → Automatically fit columns
minmax()  → Minimum 250px, maximum equal width
```

Result:

- Mobile → 1 column
- Tablet → 2-3 columns
- Desktop → More columns

Without media queries.

Grid Template Areas

```
.layout {
  display:grid;

  grid-template-areas:
"header header"
"sidebar main"
"footer footer";

  grid-template-columns:200px1fr;
}

.header {
  grid-area:header;
}

.sidebar {
  grid-area:sidebar;
}

.main {
  grid-area:main;
}

.footer {
  grid-area:footer;
}
```

Mobile Layout

```
@media (max-width:768px) {
  .layout {
    grid-template-areas:
"header"
"main"
"sidebar"
"footer";

    grid-template-columns:1fr;
  }
}
```

7. Responsive Images

Basic Image Fix

```
img,
video {
  max-width:100%;
  height:auto;
  display:block;
}
```

This fixes most overflow issues.

`object-fit`

```
.profile-pic {
  width:200px;
  height:200px;

  object-fit:cover;
  object-position:center;
}
```

Values

Value	Meaning
`cover`	Fill container without distortion
`contain`	Show full image
`fill`	Stretch image

8. Modern CSS Functions

`clamp()`

```
.heading {
  font-size:clamp(1.5rem,5vw,3rem);
}
```

Meaning

```
Minimum   → 1.5rem
Preferred → 5vw
Maximum   → 3rem
```

Fluid typography in one line.

Fluid Spacing

```
.section {
  padding:clamp(2rem,5vw,6rem);
}
```

Fluid Width

```
.container {
  width:clamp(300px,80%,1200px);
  margin-inline:auto;
}
```

`min()` and `max()`

```
.container {
  width:min(90%,1200px);
}
```

```
.image {
  width:max(300px,50%);
}
```

`calc()`

```
.sidebar {
  width:calc(100vw-250px);
}
```

Grid Calculation Example

```
.grid-item {
  width:calc((100%-2rem)/3);
}
```

Common Mistakes

1. Fixed Widths

Wrong:

```
width: 1200px;
```

Correct:

```
max-width: 1200px;
width: 100%;
```

2. Forgetting Viewport Meta Tag

Without viewport meta tag, responsiveness breaks.

3. Fixed Heights

Wrong:

```
height: 500px;
```

Better:

```
min-height: 500px;
```

4. Hover-Only UI

Mobile devices do not support hover properly.

Always ensure touch-friendly buttons.

---

5. Testing Only on DevTools

Always test on real devices too.

TL;DR

1. Use `rem` for fonts
2. Use `%`, `fr`, `vw` for layouts
3. Add viewport meta tag
4. Prefer mobile-first design
5. Use Flexbox for 1D layouts
6. Use Grid for 2D layouts
7. Use `clamp()` for fluid responsive sizing
8. Test on real devices